chiselWare

# Dff
# Product User Guide

https://github.com/chiselware/00-000-dff

November 7, 2026

# Contents

**1 Errata and Known Issues** **3**
1.1 Errata . . . 3
1.2 Known Issues . . . 3

**2 Port Descriptions** **4**

**3 Parameter Descriptions** **5**

**4 Theory of Operations** **6**
4.1 Introduction . . . 6
4.2 Interface Timing . . . 7

**5 Simulation** **8**
5.1 Tests . . . 8
5.2 Code coverage . . . 8
5.3 Running simulation . . . 8

**6 Synthesis** **9**
6.1 Area . . . 9
6.2 SDC File . . . 9
6.3 Timing . . . 9
6.4 Multicycle Paths . . . 9

# 1 Errata and Known Issues

## 1.1 Errata

None.

## 1.2 Known Issues

None.

# 2 Port Descriptions

The ports for **Dff** are shown below in Table 1. The width of several ports is controlled by the following parameter:

- *width* is the width the d and q ports in bits

| **Port Name** | **Width** | **Direction** | **Description** |
|---|---|---|---|
| clock | 1 | Input | Positive edge clock |
| reset | 1 | Input | Active high reset |
| enable | 1 | Input | Enable data |
| d | *width* | Input | Input to D flip-flop |
| q | *width* | Output | Output of D flip-flop |

Table 1: Port Descriptions

# 3 Parameter Descriptions

The parameters for **Dff** are shown below in Table 2.

| Name | Type | Min | Max | Description |
|---|---|---|---|---|
| width | Int | 1 | $\geq 1$ | Specifies the width of the D flip-flop |

Table 2: Parameter Descriptions

The Dff is instantiated into a design as follows:

```
// Instantiate small register made of flip-flops
val myRegister = Dff(width = 8)
```

# 4 Theory of Operations

## 4.1 Introduction

The **Dff** is a basic D flip-flop with a parameterized width.

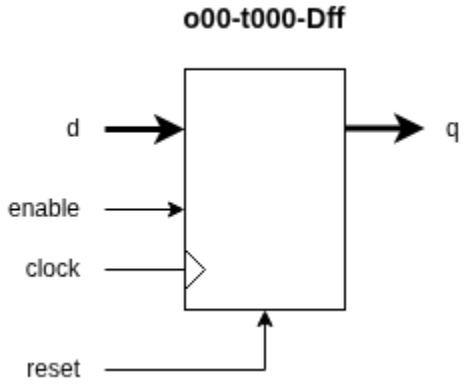



Figure 1: Block Diagram

## 4.2 Interface Timing

Data on the input **d** will appear on the output **q** on the following clock cycle if **enable** is asserted. If **enable** is not asserted, **q** will retain its current state on the next clock cycle. The timing diagram shown below in Figure 2 represents an instantiation with the following parameters.

```
val myRegister = Dff(width = 8)
```

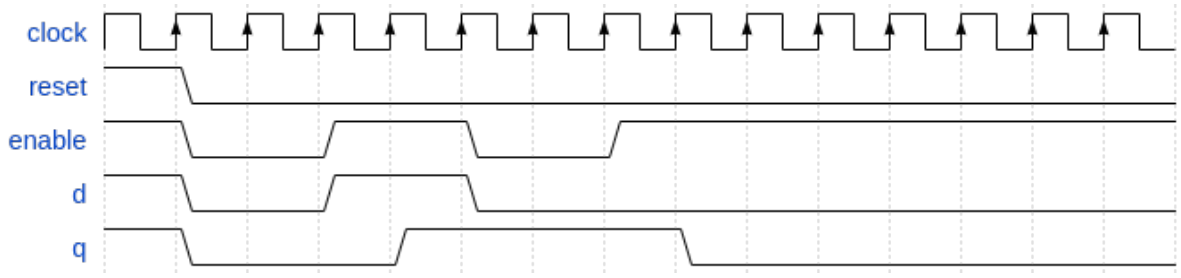



Figure 2: Timing Diagram

# 5 Simulation

## 5.1 Tests

The included test bench tests 3 configurations of the Dff. More configurations can be added as desired. For each configuration, random data is applied to the inputs and outputs are checked for correctness.

## 5.2 Code coverage

The tests exceed the chiselWare minimum standard of 90% Statement coverage and 95% Branch coverage.

## 5.3 Running simulation

Simulations can be run directly from the command prompt as follows:

```
$ sbt "project core" test
```

or from make as follows:

```
$ make test
```

# 6 Synthesis

## 6.1 Area

The **Dff** has been tested in a number of configurations and the following results should be representative of what a user should see in their own technology.

| Config Name | width | Gates |
|---|---|---|
| synW1 | 1 | 8 |
| synW64 | 64 | 597 |
| syn128 | 128 | 1194 |

Table 3: Synthesis results

## 6.2 SDC File

An `.sdc` file is generated to provide synthesis and static timing analysis tools guidance for synthesis.

The `Dff.sdc` file can be found in the test case directory as follows:
`./modules/dff/generated/synTestCases/<test case name>/dff.sdc`

## 6.3 Timing

The following timing was extracted using the generated .sdc files with the open-source Nangate 45nm library.

| Config Name | Period | Duty Cycle | Input Delay | Output Delay | Slack |
|---|---|---|---|---|---|
| synW1 | 5ns | 50% | 20% | 20% | 3.88 (MET) |
| synW64 | 5ns | 50% | 20% | 20% | 3.77 (MET) |
| synW128 | 5ns | 50% | 20% | 20% | 3.63 (MET) |

Table 4: Static Timing Analysis results

## 6.4 Multicycle Paths

None.